

SINIF VE NESNE

Sınıf ve Nesne Nedir?

Nasıl ki dünyamızda evler bir **mimari plan** (blueprint) üzerinden inşa ediliyor, **nesneler** (object) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (state) ve nesnelerin göstereceği **davranışlarının** (behavior) tanımlandığı bu plana **sınıf** (class) adı verilir. **Durumlara** (states) ilişkin veriler, **alanlarda** (fields) tutulur.

Nesne yönelimli programlamada, altyapının bize sunduğu sınıfları veya nesneleri kullanarak programı geliştiremeyiz. Mutlaka programcı tarafından ihtiyaçlara göre veri içeren ve davranış gösteren nesnelere ihtiyaç duyarız. İşte bu nesneler, **kullanıcı tanımlı sınıflar** (user defined class) ile tanımlanır.

Bir mimari plandan birçok ev yapılabileceği gibi, bir sınıftan da birçok nesne imal edilebilir. İmal edilen her **nesne** (object) sınıfın bir **örneğidir** (instance). Kullanıcı tanımlı sınıflar, en basit şekilde aşağıdaki gibi tanımlanır;

```
[erişim-değiştiricisi] class sınıf-kimliği {
    [erişim-değiştiricisi] veri-tipi alanKimliği;
    [erişim-değiştiricisi] veri-tipi davranışKimliği() { };
}
```

Sözde kodu verilen sınıf tanımındaki kimlikler için değişken kimliklendirme kuralları geçerlidir. Tanımlanan bir sınıftan nesneler **new** anahtar kelimesiyle dinamik olarak imal edilir. Yani nesne **çalıştırma anında** (run time) bellekte oluşur.

Şimdi ortak davranış ve özelliklerin tanımlandığı bir **Öğrenci** sınıfı ve bu sınıftan nesne imal eden **Program.cs** sınıfını kodlayalım.

```
// See https://aka.ms/new-console-template for more information

Öğrenci ali = new Öğrenci(); /* Öğrenci sınıfından ali nesnesi new kelimesiyle imal edildi.*/
ali.yaş = 40; /* "ali" nesnesinin "yas" alanı (field) 40 olarak değiştirildi.*/
ali.yaşınıSöyle(); /* "ali" nesnesine yaşınıSöyle mesajı verildi: Yaşım:40 */
ali.cinsiyetiniSöyle(); /* "ali" nesnesine cinsiyetiniSöyle mesajı verildi:
                        Cinsiyetimi Söylemek İstemiyorum! */

class Öğrenci // Öğrenci sınıfı
{
    public uint yaş; // yaş durumuna ilişkin verileri tutacak alan (field)
    public char cinsiyet; // cinsiyet durumuna ilişkin verileri tutacak alan (field)
    public void yaşınıSöyle() // yaşınıSöyle davranışını gösterecek yöntem
    {
        Console.WriteLine($"Yaşım:{yaş}");
    }
    public void cinsiyetiniSöyle() // cinsiyetiniSöyle davranışını gösterecek yöntem
    {
        if (cinsiyet == 'E' || cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (cinsiyet == 'K' || cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}
```

Programda **class Öğrenci** kodu ile bir Öğrenci sınıfı tanımlanmıştır. Sonrasında **Öğrenci** sınıfından imal edilecek nesnelerin cinsiyet ve yaş bilgilerini tutacak **durumlar** (state) için **cinsiyet** ve **yaş** alanı tanımlanmıştır. Bir sınıftan imal edilecek nesnelerin umuma açık davranış ve özellikleri **public** erişimiyle tanımlanır.

Daha sonra Program sınıfı Öğrenci sınıfından nesne imal edip kullanması için değiştirilecektir. Bir sınıftan nesne imal etme **new** anahtar kelimesiyle olur. Main yönteminde imal edilen **ali** nesnesinin **yaş** alanı **public** oluşu için değiştirilebilmiştir. Ayrıca **ali** nesnesine **yaşınıSöyle** ve **cinsiyetiniSöyle** davranışlarını göstermesi için **ileti gönderilmiştir** (message-passing). Burada şuna dikkat etmek gerekir nesnenin alanları değiştiğinde yani **durumları** (state) değiştiğinde davranışların da bu durumdan etkilendiği gözden kaçırılmamalıdır.

Sınıftan nesne imal etme işlemine **örnekleme** (instantiation) adı verilir. Her bir nesne sınıfın bir **örneğidir** (instance). Örnek üyelerine yukarıda görüldüğü üzere **nokta işleci** (dot operator) ile erişilir.

Nesnelerin her biri;

1. **Öbek** (heap) bellek yer kaplar.
2. Bir **kimliğe** (identity) sahiptir.
3. Verileri tutan **durumlara** (state) sahiptir.
4. **Özelliklere** (property) sahiptir. Nesne, durumlarına değer atamayı veya nesne verilerini dışarı aktarmayı kontrol eder. Yani verileri kendi mülküdür.
5. **Davranışlara** (behavior) sahiptir. Nesnenin verilerine göre farklı yöntemlere davranış gösterirler.
6. **Roller** (role) veya bir başka adıyla **ara yüzlere** (interface) sahiptir. Nesneler kullanıldıkları yerlerdeki rollerine göre çalışır.
7. **Olaylardan** (event) etkilenirler.

Erişim Değiştiricileri

Sınıftan imal edilen nesnelerin üyelerine başka nesne ve programların nasıl erişeceği **erişim değiştiricileri** (access modifiers) ile belirlenir. **Görünürlük** (visibility) sıfatları olarak da adlandırılan bu değiştiriciler kısaca sınıf üyelerinin sınıf içinden ve dışından erişimi belirleyen sıfatlardır. Bunlar;

- **public**: Erişimi umuma açık olarak belirlenen durum ve davranışlara diğer bütün bileşenlerden erişilir.
- **private**: Erişimi mahrem/şahsi/kişisel olarak belirlenen durum ve davranışlara sadece içinde bulunulan sınıf içerisinden erişilebilir.
- **protected**: Erişimi korumalı olarak belirlenen durum ve davranışlara sadece içinde bulunulan sınıf ve türemiş sınıflar içerisinden erişilebilir. Bu belirleyici ileride *Kalıtım* başlığında incelenecektir.
- **internal**: Erişimi dahili olarak belirlenen durum ve davranışlara sadece içinde bulunulan **montaj** (assembly) içinden erişilebilir.
- **protected internal**: Erişimi dahili korumalı olarak belirlenen durum ve davranışlara içinde bulunulan sınıf ve türemiş sınıflar veya içinde bulunulan **montaj** (assembly) içinden erişilebilir.
- **private protected**: Erişimi mahrem korumalı olarak belirlenen durum ve davranışlara içinde bulunulan sınıf veya sınıftan türemiş sınıflar veya içinde bulunulan **montaj** (assembly) içinden erişilebilir.
- **file**: Erişimi dosya olarak belirlenen durum ve davranışlara içinde bulunulan kaynak kod dosyası içinden erişilebilir.

```
// See https://aka.ms/new-console-template for more information
Kişi ahmet = new Kişi(); /* Kişi sınıfından ahmet nesnesi new kelimesiyle imal edildi.*/
ahmet.adı = "Ahmet"; /* "ahmet" nesnesinin "adı" alanı değiştirilebildi.
                        Çünkü bu alan umuma açık (public) tanımlı. */
ahmet.adınıSöyle(); /* "ahmet" nesnesine adını söyle davranışını
                    göstermesi için mesaj verildi (message-passing):
                    Adım:Ahmet */
//ali.sırDeğiştir(); /* Bu yöntem çalışmaz! Çünkü mahrem (private) olarak tanımlanmış. */

class Kişi // Kişi sınıfı
{
    public string adı; /* "Adı" kimlikli alan(field),
                       imal edilecek nesnelerinin adına ait
                       durumlara(state) ilişkin verileri tutar. */
    private string sır; /* "sır" kimlikli alan(field),
                       imal edilecek Kişi nesnelerinin
```

```

        sırrına ait durumlara(state)
        ilişkin verileri tutar. */
    public void sırDeğiştir() /* sırDeğiştir() yöntemi(method),
        imal edilecek Kişi nesnelerinin
        sırrını depğıştirme
        davranışını(behavior) tanımlıyor */
    {
        sır = "Gizli bir şey öğrendim!";
    }
    public void adınıSöyle() /* adınıSöyle() yöntemi(method),
        imal edilecek Kişi nesnelerinin
        adını söyleme davranışını(behavior) tanımlıyor */
    {
        Console.WriteLine($"Adım:{adı}");
    }
}

```

Yukarıda erişim belirleyicilerin uygulandığı basit bir **Kişi** sınıfı örnek verilmiştir. Bu sınıftan imal edilen nesnelerin **adı** özelliği ve **adınıSöyle** davranışına diğer tüm nesneler tarafından erişilip değiştirilebilir, ancak **sır** özelliğine ve **sırDeğiştir** davranışına ise hiçbir nesne tarafından erişilemez.

Yöntemler

Sınıf tanımında görüldüğü üzere yöntemler imal edilecek nesnelerin davranışlarıdır. Yöntemler yapısal programlamadaki fonksiyonlar gibi kodlanır; *Yapısal Programlama* mantığında fonksiyonlar içinde ilk önce değişkenler tanımlanır ve daha sonra bu değişkenlerde tutulan veriler kontrol işlemleri ile işlenir.

Yöntem Tanımı

C# dilinde yöntemler aşağıdaki gibi tanımlanır;

```

Erişim-Değiştirici Seçimlik-Erişim-Değiştirici Dönüş-tipi Yöntem-kimliği(Parametre-Listesi)
{ //Yöntem Blok Başlangıcı
    /*
        Yöntem Gövdesi
        [return dönüş-tipinde-değer;]
    */
} //Yöntem Bloğu Bitişi

```

Yöntemin tanımı; girdileri yani parametreleri, yaptığı işlem ve döndüreceği değeri içerecek şekilde başlık ve gövdesinin kodlanmasından oluşur.

- Yöntemler, **sınıf (class)** ve ileride göreceğimiz **yapı (struct)** ve **ara yüz (interface)** içinde **erişim değiştiriciler (access modifier)** ile tanımlanır.
- Seçimlik erişim değiştiricileri **statik**, **override**, **virtual**, **abstract**, **new** ve **sealed** olabilir. İleride anlatılacaktır.
- Yöntem adı, fonksiyonun benzersiz kimliğidir ve kimliklendirmede, değişken kimliklendirme kuralları geçerlidir.
- Bağımsız değişkenler, yöntemlere giren parametrelerdir. Parametre listesi, bir işlevin parametrelerinin veri tipini, sırasını ve sayısını belirtir. Parametreler isteğe bağlıdır; yani bir fonksiyon hiçbir parametre içermeyebilir.
- Parametre listesi değişken bildirimlerine benzer ve virgül işlecisi ile birbirlerinden ayrılırlar; **[veritipi parametre1][,veritipi parametre2][,veritipi parametre3]...**
- Yöntemin gövdesi, yöntemin ne yaptığını tanımlayan talimatları içerir. Yapısal programlamadaki gibi ilk önce değişkenler tanımlanmalı daha sonra bu değişkenleri işleyecek talimatlar yazılmalıdır. Yöntemde istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani algoritma belirlenir.
- Yöntemler bir değer döndürebilir. Dönüş tipi, yöntemin döndürdüğü değerin veri tipidir. Dönüş tipine uygun olarak bir değer, **return** talimatı ile geri döndürülmelidir. **void** veri tipi hiçbir değer döndürmeyen yöntemlerin geri dönüş tipidir. Bu durumda yöntemden geri dönmek için **return** kelimesi tek başına kullanılabilir.

- **return** talimatı, bir fonksiyonun yürütülmesini sonlandırır ve kontrolü çağırdığı yere geri verir. Yani programın icrasına devam etmek için çağırdığı yere dönlür.
- Yöntemin çağrıldığı yerde, geri döndürülen değerin tipi ile eşleşen bir değişkene dönüş değeri atanabilir.

Yöntem Çağırma

Statik Üyeler ve Statik Sınıf başlığında detayı verilmiştir. Şu ana kadar çok kullandığımız **statik** bir yöntemler aşağıdaki gibi çağrılır.

```
System.Console.WriteLine("Merhaba");// Tek argüman
string adı = "Kullanıcı";
System.Console.WriteLine("Merhaba, {0}!", adı); // Çoklu Argüman
string input = System.Console.ReadLine(); // Argümansız string geri döndüren.
```

İmal edilmiş nesnelerde yani **örnek** (*instance*) yöntemi nokta işleci ile aşağıdaki gibi çağrılır;

```
Kişi ahmet = new Kişi();
ahmet.adınıSöyle();
Hesap h = new Hesap();
h.örnek();
```

Aşağıda **topla** yöntemine sahip bir **Hesap** sınıfı ve bu yöntemin kullanılması örnek verilmiştir.

```
// See https://aka.ms/new-console-template for more information
Hesap h = new Hesap(); // Hesap sınıfından h adında bir nesne oluşturuldu
Console.WriteLine(h.topla(20, 30)); /* h nesnesine topla mesajı verildi.
                                     topla yönteminin geri döndürdüğü değer konsola
yazdırıldı. */
h.örnekhesap(); /* h nesnesine örnekhesap mesajı verildi.
                 örnekhesap yönteminin geri döndürdüğü değer konsola yazdırıldı. */

class Hesap // Hesap Sınıfı
{
    public int topla(int p1, int p2)
    {
        return p1 + p2;
    }
    public void örnekhesap()
    {
        topla(1, 2);
        /* topla fonksiyonu 1 ve 2 argümanlarıyla çağrılıyor.
         * Ama geri döndürdüğü değer kullanılmıyor */
        Console.WriteLine(topla(3, 4));
        /* 3 ve 4 argümanlarıyla çağrılan topla
         * yönteminin geri döndürdüğü değer
         * konsola yazdırıldı */
        int toplam;
        toplam = topla(5, 6);
        Console.WriteLine(toplam);
        /* topla yönteminin geri döndürdüğü değer
         * toplam yerel değişkenine atandı */
        Console.WriteLine(topla(7, 8) + topla(9, 10));
        /* topla yönteminin geri döndürdüğü değer ile
         * bir başka topla yönteminin geri döndürdüğü değer
         * toplanarak konsola yazdırıldı */
        Console.WriteLine(topla(10, topla(11, 12)));
        /* topla yönteminin argüman olarak
         * bir başka topla yönteminin geri döndürdüğü değer
         * geçirildi ve konsola yazdırıldı */
    }
}
```

Yöntemlere Parametre Geçirme

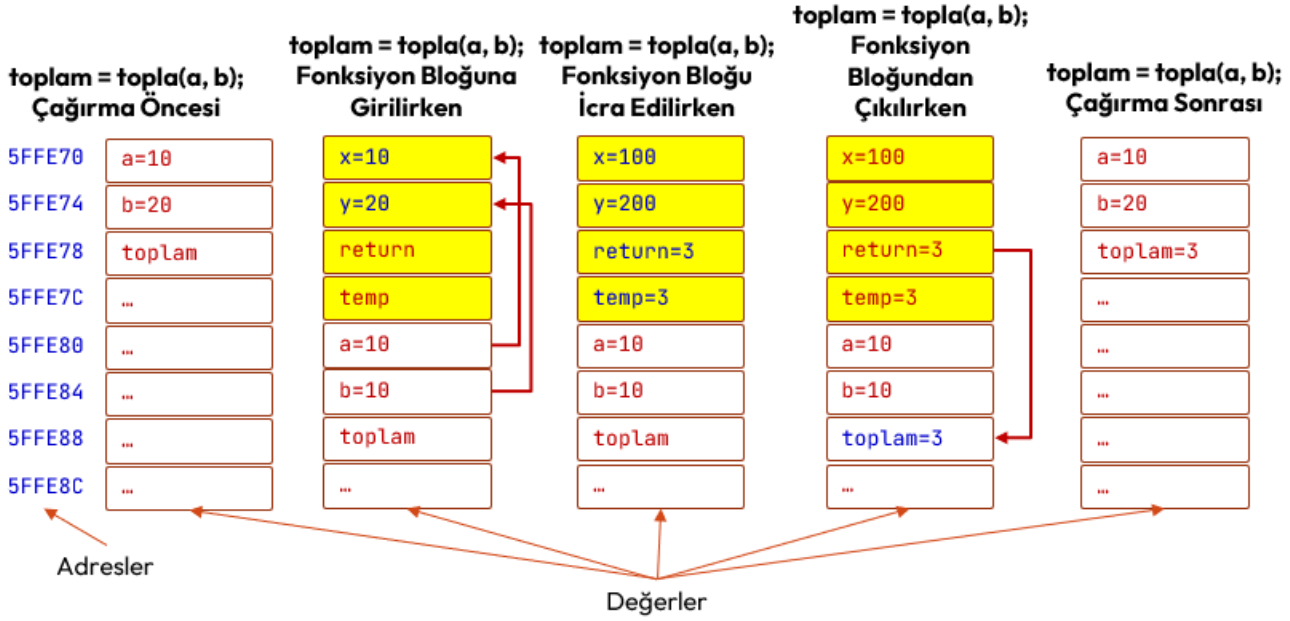
Bir yönteme değer tip parametre olarak geçirildiğinde **çağrı** (call/invoke) sırasında bellekteki değişim aşağıdaki programdan anlaşılabilir;

```
// See https://aka.ms/new-console-template for more information
int a = 10;
int b = 20;
int toplam = 0;
Hesap hesap = new Hesap(); //Hesap sınıfından bir hesap nesnesi imal ediliyor.
Console.WriteLine("çağrı öncesi a: {0}, b:{1}", a, b);
//çağrı öncesi a: 10, b:20
toplam = hesap.topla(a, b);
Console.WriteLine("çağrı sonrası a: {0}, b:{1}, toplam:{2}", a, b, toplam);
//çağrı sonrası a: 10, b: 20, toplam: 30

public class Hesap
{
    public int topla(int x, int y)
    { /* iki tamsayıyı toplayan fonksiyon tanımı*/
        /* Topla fonksiyonu yerel değişkenler */
        int temp = x + y;
        x = 100; y = 200;
        /*
            Burada değiştirilenler parametre değişkenleridir.
            main() içindeki yerel değişkenler değildir.
        */
        Console.WriteLine("çağrı içinde x: {0}, y:{1}", x, y);
        //çağrı içinde x: 100, y:200
        return temp;
    }
}
```

Yukarıdaki kod incelendiğinde;

- İcra sırası **topla(a,b)** fonksiyonuna geldiğinde fonksiyon çağrılmadan önce main fonksiyonu yerel değişkenleri a, b ve toplam değişkenleri yığın (stack) belleğe itilir (push).
- **topla(a,b)** fonksiyonu çağrıldığı anda fonksiyon bloğuna başlamadan parametrelere sırasıyla a ve b değişkenlerinin değerleri kopyalanarak fonksiyona geçirilir.
- **topla(a,b)** fonksiyonu icra edilirken fonksiyon yerelindeki **temp** ve parametre değişkenleri **x**, ve **y** istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- **toplam=topla(a,b)** fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki toplam değişkenine atanır ve fonksiyon için yığın (stack) belleğe itilen değerler geri çekilir.



Şekil 22. Fonksiyon çağırma sürecinde yığın bellek

Yukarıda örneği verilen programda fonksiyon çağırma sürecindeki **yığın bellek** (stack segment) değişimi gösterilmektedir. Bu örnekte olduğu gibi değer tiplerin parametre olarak bir fonksiyona geçirildiğinde yığın bellekte bu değerlerin bir kopyası oluşur ve yöntem içinde bu kopyalar kullanılır. Bu nedenle, yöntem içinde değer tip olarak geçirilen parametrede yapılan değişikliklerin argüman üzerinde hiçbir etkisi yoktur.

Bir yönetime **değer tipi parametre olarak geçirirsek** (pass by value);

- Değer parametreleri değişkenlerin bir kopyasını oluşturur, bu nedenle yöntem içindeki herhangi bir değişiklik orijinal değeri değiştirmez. Ayrıca **değişmezleri** (literal) de yönetime parametre olarak geçirme imkânımız olur. Bu, orijinal verilerin değişmeden kalması gerektiğinde yararlıdır.
- Değer parametreleri orijinal değışkende değişiklik yapılmasını engeller. Orijinal değerleri değişmeden tutmak istediğinizde değer parametrelerini kullanabilirsiniz.
- Bellekte az yer kaplayan veri tipleri için değer parametreleri kullanabilirsiniz çünkü bunların kopyalanması kolaydır ve performansı etkilemez.

Yöntemlere Referans Olarak Parametre Geçirme

Bir referans değişken, bir değişkenin değerine değil bellek konumuna bir referanstır. Bir yönetime **referans veri tipleri** ya da **bir referansı parametre olarak geçirdiğinizde** (pass by reference), değer parametrelerinin aksine, bu parametrelerin değerleri için yeni bir depolama konumu oluşturulmaz. Referans parametreleri, yönetime sağlanan gerçek parametrelerle aynı bellek konumunu temsil eder. Böylece parametre değerleri üzerinde değişiklik yapma imkânımız olur. Ancak değer tipleri için durum farklıdır. Değer tipindeki parametreleri yöntem içerisinde değiştirmek istiyorsak önünde **ref** anahtar kelimesi kullanmalıyız.

```
// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlemci = new Sayıİşlemleri(); // DeğerDeğiştir metodu için bir nesne oluşturduk
int a = 10;
int b = -10;
Console.WriteLine("DeğerDeğiştir, öncesi a:{0}, b:{1}", a, b); // a:10 b:-10
işlemci.DeğerDeğiştir(ref a, ref b);
Console.WriteLine("DeğerDeğiştir sonrası a:{0}, b:{1}", a, b); // a:-10 b:10

class Sayıİşlemleri
{
    public void DeğerDeğiştir(ref int x, ref int y)
    {
        int temp;
        temp = x;
```

```

        x = y;
        y = temp;
    }
}

```

Bir yönteme referans veri tipleri ya da **bir referansı parametre olarak geçirdiğinizde** (pass by reference);

- Orijinal değişkenleri değiştirme imkânımız olur.
- Hafızada çok yer kaplayan parametreler için çok kullanışlıdır.

Yöntem Parametrelerini Çıktı Olarak Kullanma

Bunların dışında yöntemdeki **return** ifadesi, bir fonksiyondan yalnızca bir değer döndürmek için kullanılabilir. Ancak, **out** parametrelerini kullanarak bir fonksiyondan birden fazla değer döndürebilirsiniz. **out** Parametreleri, referans parametrelerine benzerdir, ancak veriyi yönteme aktarmak yerine yöntemden dışarı aktarırlar.

```

// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlem = new Sayıİşlemleri();
int okunan, okunanınKaresi;
okunan = işlem.birSayı0ku(out okunanınKaresi);
Console.WriteLine($"Okunan: {okunan}, karesi:{okunanınKaresi}");

class Sayıİşlemleri
{
    public int birSayı0ku(out int kare)
    {
        int temp;
        Console.WriteLine("Bir sayı giriniz: ");
        temp = Convert.ToInt32(Console.ReadLine());
        kare = temp * temp;
        return temp;
    }
}

```

Ön Tanımlı Argüman

Argümanlar fonksiyonun çağrıldığı anda fonksiyona geçirilen gerçek parametrelerdir. Yöntemlere de fonksiyonlar gibi parametre geçirilebilir. Yukarıda verilen **topla** yönteminde iki farklı **int** parametresi aldığı görülmektedir. İstenirse bu parametrelere ön tanımlı bir değer atanabilir yani yöntemler **ön tanımlı argümanlar** (default argument) ile tanımlanabilir. Bu şekilde tanımlanan yöntem aşağıda örneği verildiği üzere **topla(10)**; şekilde çağrılabilir,

```

// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlem = new Sayıİşlemleri();
int sonuç= işlem.topla(10); // 10 + 1 = 11
Console.WriteLine($"Sonuç: {sonuç}"); //11

class Sayıİşlemleri
{
    public int topla(int p1, int p2 = 1) //İkinci parametre öntanımlı olarak 1
    {
        return p1 + p2;
    }
}

```

Özyinelemeli Yöntemler

Özyineleme (recursion), bir yöntemin kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, **yineleme** (iteration), **döngüler** (loop) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. **Özyinelemeli** (recursive)

algoritmalarla, tekrarlar yöntemin kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır.

1. Cevabı kesin olarak bildiğimiz **temel durum** (base case)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n 'nin faktöriyeli, $n!$ ile gösterilir ve n 'den küçük veya ona eşit olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 3 \cdot 2 \cdot 1$$

Negatif olmayan bir tam sayı olan n 'nin faktöriyeli aynı zamanda n 'nin bir sonraki daha küçük faktöriyelle çarpımına eşittir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \cdot (n - 1)!$$

Örneğin;

$$5! = 5 \cdot 4! = 5 \cdot 4 \cdot 3! = 5 \cdot 4 \cdot 3 \cdot 2! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir yöntem yazılmıştır.

```
// See https://aka.ms/new-console-template for more information
Faktöriyel f = new Faktöriyel();
Console.WriteLine(f.faktöriyelHesapla(5)); // 120
class Faktöriyel
{
    public uint faktöriyelHesapla(uint n)
    {
        uint sonuc = 1, sayaç;
        for (sayaç = n; sayaç > 0; sayaç--)
            sonuc = sonuc * sayaç;
        return sonuc;
    }
}
```

Sıfır sayısı da pozitif bir sayıdır ama sonucum sıfır çıkmaması için 0 sayısının faktöriyeli 1 kabul edilir. Bu aslında faktöriyel problemi için **temel durumdur** (base case) ve **boş ürün** (empty product) kuralı olarak bilinir. Kısaca faktöriyel fonksiyonu aşağıdaki şekilde gösterilir;

$$n \in \mathbb{N} \text{ olmak üzere } f(n) = n! = \begin{cases} 1, & n = 0 \\ n \cdot f(n - 1), & n > 0 \end{cases}$$

Özyinelemeli (recursive) olarak ise bu yöntem aşağıdaki şekilde kodlanabilir;

```
public uint faktöriyelHesapla(uint n)
{
    return n == 0 ? 1 : n * Factorial(n - 1);
}
```

Özyinelemeli olarak Fibonacci sayısı aşağıdaki gibi hesaplanabilir;

```
public int fibonacci(int i) {
    if(i <= 2) return 1;
    return fibonacci(i - 2) + fib(i - 1);
}
```

Özyinelemeli olarak bir tamsayının kuvveti aşağıdaki gibi hesaplanabilir;

```
public int kuvvet(int taban, int üs) {
    if (taban == 0) return 0;
    if (üs == 0) return 1;
    return taban * kuvvet(taban, üs - 1);
}
```

Parametre Olarak Diziler

Yöntemlere parametre olarak gönderilen dizinin boyutu belirten köşeli parantez açılır ve kapatılır. Bu durumda tek boyutlu diziler parametre olacak ise dizinin boyutu yazılmayabilir.

Aşağıda dizileri parametre olarak alan fonksiyon bildirim örnekleri bulunmaktadır;

```
void diziIsleyenFonksiyon1(int[] tekBoyutluDizi);
void diziIsleyenFonksiyon2(int[,] matris);
void diziIsleyenFonksiyon3(int[,,,] üçBoyutluDizi);
```

Aynı fonksiyonları aşağıdaki şekilde çağırabiliriz;

```
int[] notlar=new int[10];
int[,] dersNotlari=new int[2,10]; //2 dersi alan 10 öğrenci için notlar;
int[,,,] bolumDersNotlari=new int[4,2,10]; //4 bölümde 2 dersi alan 10 öğrenci için notlar;
//...
diziIsleyenFonksiyon1(notlar);
diziIsleyenFonksiyon2(dersNotlari);
diziIsleyenFonksiyon3(bolumDersNotlari);
//...
```

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```
// See https://aka.ms/new-console-template for more information
int[] dizi = { 1, 2, 3, 4, 5 };
Diziİşlemleri işlemler = new Diziİşlemleri();
Console.WriteLine("Dizi:");
işlemler.diziyaz(dizi);
// 1 2 3 4 5
Console.WriteLine();
Console.WriteLine("Ortalama: {0}", işlemler.ortalama(dizi));
// Ortalaması: 3
int[,] matris = new int[4, 4]{ { 1, 2, 3, 4},
                              { 5, 6, 7, 8},
                              { 9, 10, 11, 12},
                              {13, 14, 15, 16} };

Console.WriteLine("Matris Önce:");
işlemler.matrisyaz(matris);
/*
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
*/
işlemler.transpose(matris);
Console.WriteLine("Matris Sonra:");
işlemler.matrisyaz(matris);
/*
1 5 9 13
2 6 10 14
3 7 11 15
4 8 12 16
*/

class Diziİşlemleri
{
    public float ortalama(int[] dizi)
    {
        return (float)dizi.Average();
    }
    public void diziyaz(int[] dizi)
    {
        for (int i = 0; i < dizi.Length; i++)
```

```

    {
        Console.Write($"{dizi[i],4}");
    }
}
public void transpose(int[,] dizi)
{
    for (int i = 0; i < dizi.GetLength(0); i++)
    {
        for (int j = i; j < dizi.GetLength(1); j++)
        {
            int temp = dizi[i, j];
            dizi[i, j] = dizi[j, i];
            dizi[j, i] = temp;
        }
    }
}
public void matrisyaz(int[,] dizi)
{
    for (int i = 0; i < dizi.GetLength(0); i++)
    {
        for (int j = 0; j < dizi.GetLength(1); j++)
        {
            Console.Write($"{dizi[i, j],4}");
            Console.WriteLine();
        }
    }
}
}

```

Çağrı Kuralı

Çağrı kuralı (**calling convention**), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya göre iletileceğini belirtir. Birincisi C# dilinde kullanılır ve argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. İkincisi ise C/C++ dilinde kullanılır ve argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir.

```

// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlem = new Sayıİşlemleri();
int sonuç = işlem.topla(10,20,30); //p1:10,p2:20,p3:30
Console.WriteLine($"Sonuç: {sonuç}"); // Sonuç: 60
int a = 1;
int toplam = işlem.topla(a, ++a, a++);
/* Beklenen Sonuç: 1+2+2=5
 * Parametreler: p1:1,p2:2,p3:2 */
Console.WriteLine($"Sonuç: {toplam}"); // Sonuç: 5

class Sayıİşlemleri
{
    public int topla(int p1, int p2, int p3)
    {
        Console.WriteLine($"p1:{p1},p2:{p2},p3:{p3}"); // p1: 1
        return p1 + p2+ p3;
    }
}

```

Yukarıdaki örnek incelendiğinde **sonuç1** ve **sonuc2** argümanlarının hangi sırada fonksiyona geçirildiğinin bir önemi yoktur. Ancak aşağıda verilen C++ kodu örneği farklı çıktı üretecektir.

```

#include <iostream>
using namespace std;
int topla(int, int, int);
int main () {
    int a = 1,toplam;
    toplam = topla(a, ++a, a++);
    cout << "toplam:" << toplam << endl;
}

```

```

}
int topla(int x, int y, int z) {
    cout << "x:"<< x << " y:" << y << " z:" << z<< endl;
    return x+y+z;
}
/* Program çalıştırıldığında:
x:3 y:3 z:1
toplam:7
*/

```

Bu kodun çıktısı **1 2 3** olarak beklenir ancak çağrı kuralından ötürü çıktı **3 3 1** olur. Bunun nedeni C++ dilinin çağrı kuralının sağdan sola olmasıdır. Bunu şöyle açıklayabiliriz;

1. Fonksiyona sağdan ilk argüman olarak a değişkeni değeri 1 geçirilir.
2. Ardından a++ ifadesi ile a değişkeni 2'ye yükseltilir.
3. Sonra ++a ifadesi ile a değişkeni 3'e yükseltilir.
4. Fonksiyona ikinci argüman olarak 3 geçirilir.
5. Fonksiyona son olarak a'nın son değeri olan 3 geçirilir.

Çağrı kuralı C++ dillerinde yazılmış bir kütüphanedeki fonksiyonları projemizde kullanmak istediğimizde önem kazanır¹⁸.

Nesne Yapıcısı

Bir mimari plandan birçok ev yapılabileceği gibi, bir sınıftan birçok nesne imal edilebilir. İmal edilen her **nesne** (**object**) sınıfın bir **örneğidir** (**instance**). İmal edilecek nesnelerin **ön tanımlı** (**default**) olarak belirlenen **durumlarla** (**state**) imal edilmesi gerekiyorsa nesne **yapıcısı** (**constructor**) kullanılır.

Yapıcıların görevi bir sınıftan nesneleri, varsayılan durumlara sahip olarak imal etmektir. Nesnelere ait veriler, **alanlarda** (**field**) tutulurlar ve nesnenin **durumumu** (**state**) bu veriler belirler. Çünkü durumlar nesnenin davranışını etkiler.

Nesne yapıcıları sınıf kimliğiyle aynı olup yöntemlerin aksine geri değer döndürmezler. Amacı kısaca imal edilecek nesnelerin durumlarına ilk değer vermektir.

```

// See https://aka.ms/new-console-template for more information
Öğrenci ilhan = new Öğrenci(); /* Öğrenci sınıfından "ilhan" nesnesi;
                                Öğrenci() yapıcısı kullanılarak
                                yaş durumu 18 ve
                                cinsiyet durumu E olarak imal edildi. */

ilhan.yaşınıSöyle(); // Yaşım:18
ilhan.cinsiyetiniSöyle(); // Cinsiyetim: ERKEK
ilhan.yaş = 50; /* "ilhan" nesnesinin umuma açık "yaş" durumu değiştirildi. */
ilhan.yaşınıSöyle(); // Yaşım:50
/* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message passing). */

class Öğrenci // Öğrenci sınıfı
{
    public Öğrenci() // Öğrenci sınıfından imal edilecek ön tanımlı yapıcı
    {
        /* Yapıcı içerisinde imal edilecek nesnelerde, alanlara (field) ilk
        * değer verilir. */
        yaş = 18;
        cinsiyet = 'E';
    }
    public uint yaş;
    /* "yaş" kimlikli alan(field), öğrencinin yaşına ait durumlara(state)
    * ait verileri tutar. */
    public char cinsiyet;
    /* "cinsiyet" kimlikli alan(field), öğrencinin yaşına ait durumlara(state)
    * ait verileri tutar. */
}

```

¹⁸ <https://learn.microsoft.com/en-us/dotnet/standard/native-interop/calling-conventions>

```

public void yaşınıSöyle()
{
    /* yasiniSoyle() yöntemi(method), öğrencinin yaşını söyleme
    * davranışını(behavior) tanımlıyor */
    Console.WriteLine($"Yaşım:{yaş}");
}
public void cinsiyetiniSöyle()
{
    /* cinsiyetSoyle() yöntemi(method), öğrencinin cinsiyetini söyleme
    * davranışını(behavior) tanımlıyor */
    if (cinsiyet == 'E' || cinsiyet == 'e')
        Console.WriteLine("Cinsiyetim: ERKEK");
    else if (cinsiyet == 'K' || cinsiyet == 'k')
        Console.WriteLine("Cinsiyetim: KADIN");
    else
        Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
}
}

```

Örnekte **Öğrenci** sınıfından imal edilecek her nesne ön tanımlı olarak yaşı 12, ve cinsiyeti 'E' olarak üretilmektedir. Hiçbir parametresi olamayan bu yapıcıya **ön tanımlı yapıcı** (**default constructor**) adı verilir.

Soyut, Somut ve Bilgi Gizleme

Soyut (**abstract**) kavramı kelime itibarıyla detay içermeyen, özet anlamına gelir. **Somut** (**concrete**) ise en ince detayına kadar bilinen, elle tutulur anlamına gelir. Bir şey hakkında soyutlama yapıldığında, o şeyin detayıyla ilgili olan tüm **bilgiler gizlenir** (**information hiding**).

Soyut sınıflar (**abstract class**) da tanımlanabilir ancak bu sınıflardan nesne imal edilemez. Yani bu sınıfın örneği (**instance**) olamaz. Daha çok kavramsal olarak kullanılır; Soyut bir sınıfın en az bir **soyut yöntemi** (**abstract method**) olur. Soyut sınıflar hangi davranışı göstereceğini söyler ama nasıl davranış göstereceğini anlatmaz. Bu nedenle abstract yöntem için gövde tanımlanmaz. **Somut sınıf** (**concrete class**) ile nesne imal edilebilir. **Yani örneği** (**instance**) vardır. Aşağıda soyut bir sınıf tanımı verilmiştir;

```

abstract class Soyut
{
    public abstract void Yöntem(); //Soyut Yöntemler için gövde tanımlanmaz!
}

```

Somut sınıflar ise ileride anlatılacaktır.

Bir sınıftan imal edilen nesneler tüm **durum** (**state**) veya **davranışlarını** (**behavior**) diğer nesnelere şeffaf olarak gösterip diğer nesnelerin değiştirmesine izin verilmemesi gerekir. Bu durumda nesnenin durum ve davranışları tutarsız hale gelir. Bu nedenle bir sınıftan imal edilen nesnelerin durum ve davranışlarının diğer nesnelere veya dış dünyaya kontrollü bir şekilde açılması gerekir.

Soyutlama (**abstraction**), kullanıcıya gerekli bilgileri gösterme ve kullanıcıya göstermek istemediği veya belirli bir kullanıcıyla ilgisi olmayan ayrıntıları gizleme işlemidir. Bir sınıfta soyutlama iki türlü yapılır;

- **Veri soyutlaması** (**data abstraction**): nesnenin durumları üzerinden yapılan soyutlama.
- **Kontrol soyutlaması** (**control abstraction**): nesnenin davranışları üzerinde yapılan soyutlama.

Sarmalama

Yukarıda verilen **Öğrenci** örneğinden devam edecek olursak imal edilecek nesnenin **yaş** durumuna bir başka nesne veya program **-1** gibi bir değer atayabilir. Bu durumda **yaşınıSöyle()** davranışında istenmeyen bir çıktı verilmesine sebep olur. Bunu gidermek için veri soyutlaması yapılması gerekir. Bu durumda yas alanına bir değer koymayı ve yas alanından bir değer almayı kontrol altına almalıyız. Benzer durum **cinsiyet** alanı için de geçerlidir.

Veri soyutlamasını yapmak için **yaş** ve **cinsiyet** alanlarını **mahrem** (**private**) olarak belirleyip bu alanlara değer atama ve değer okuma işlevlerini yerine getirecek **özellikler** (**property**) olan **Yaş** ve **Cinsiyet** tanımlanmıştır. Özellikler, nesnenin durumlarına kontrollü değer atayan ve değerleri kontrollü okuyan yapılardır. Özelliklerin veri tipi, kontrol edeceği **alanın** (**field**) veri tipiyle aynıdır. Bunun için **get** ve **set** kelimeleri kullanılır. Set yöntemindeki **value** atanmak istenen değeri gösteren değerdir;

```
// See https://aka.ms/new-console-template for more information
Öğrenci ilhan = new Öğrenci();
ilhan.Yaş = 50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
ilhan.yaşınıSöyle(); // Yaşım:50
ilhan.Yaş = 5; /* ilhan nesnesinin Yaş özelliği değişmez:
                Öğrenci 6 Yaşından Küçük Olamaz.*/
ilhan.Cinsiyet = 'H'; /* ilhan nesnesinin Cinsiyet özelliği değişmez:
                Cinsiyet E,e,K,k,B,b dışında bir değer olamaz. */

class Öğrenci
{
    public Öğrenci() // Öğrenci yapıcısı
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    private uint yaş;
    public uint Yaş //Yaş Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return yaş; }
        set {
            if (value < 6)
                Console.WriteLine("Öğrenci 6 Yaşından Küçük Olamaz.");
            else
                yaş = value;
        }
    }
    private char cinsiyet;
    public char Cinsiyet //Cinsiyet Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return cinsiyet; }
        set {
            if (value == 'E' |
                value == 'e' |
                value == 'K' |
                value == 'k' |
                value == 'B' |
                value == 'b')
                cinsiyet = value;
            else
                Console.WriteLine("Cinsiyet E,e,K,k,B,b Dışında " +
                                   " Bir Değer Olamaz.");
        }
    }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (cinsiyet == 'E' || cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (cinsiyet == 'K' || cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}
```

```
}
}
```

İşte bir **durumun** (**state**) veri soyutlaması yapılmış haline **özellik** (**property**) adı veririz. Çünkü artık duruma dışarıdan doğrudan müdahale edilemez. Nesnenin durum üzerinde kontrol hakları vardır.

Benzer şekilde **davranışlar** (**behavior**) da nesne dışına kontrollü olarak açılabilir. Bu durumda da **kontrol soyutlaması** (**control abstraction**) yapılır. Aşağıda faktöriyel hesaplayan bir hesap makinesinin ekranı yenileme mahrem davranışı olan **ekraniYenile()** buna örnek verilebilir;

```
// See https://aka.ms/new-console-template for more information
HesapMakinesi hesaplayici = new HesapMakinesi(); // Hesaplanan: 0
hesaplayici.Hesaplanan = 3;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 6
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 720 (6! = 720)
hesaplayici.makineyiSıfırla();
hesaplayici.Hesaplanan = 8;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 40320 (8! = 40320)

class HesapMakinesi
{
    private int hesaplanan;
    private void ekranıYenile()
    {
        Console.WriteLine($"Hesaplanan: {hesaplanan}");
    }
    public HesapMakinesi()
    { //Yapıcı
        hesaplanan = 0;
        ekranıYenile();
    }
    public int Hesaplanan
    {
        set
        {
            hesaplanan = value;
            ekranıYenile();
        }
        get
        {
            return hesaplanan;
        }
    }
    public void makineyiSıfırla()
    {
        hesaplanan = 0;
        ekranıYenile();
    }
    public void FaktöriyelHesapla()
    {
        int temp = 1;
        for (int sayac = hesaplanan; sayac > 0; sayac--)
            temp = temp * sayac;
        hesaplanan = temp;
        ekranıYenile();
    }
}
```

Bir sınıf üzerinde **veri soyutlaması** (**data abstraction**) ile veriler üzerine kılıf çekilir, hem de **kontrol soyutlaması** (**control abstraction**) ile davranışların üzerine bir kılıf çekilir. Bir sınıf üzerinde her iki işlemin birlikte yapılmasına **sarmalama** (**encapsulation**) adı verilir. Soyutlamaların gerekli olup olmadığına programcı karar verir.

Sarmalama işlemi örneklerde görüldüğü üzere **erişim değiştiricilerle** (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış olur. Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden **soyut** (**abstract**) olarak kullanmak mümkün olmaktadır. Programcı tasarladığı sınıfın soyut olarak katlanılacağını düşünerek sınıfları tasarlamalıdır.

Özetle **sarmalama** (**encapsulation**) tekniğini uygulamak, kullanımı kolay **sınıf** (**class**), **bileşen** (**component**), **kütüphane** (**library**) ve **çerçeve yapılar** (**framework**) elde etmemizi sağlar;

- n adet **veri** (**data**) ve n adet **davranış** (**behavior**) sarmalama işlemine tabi tutulursa **sınıf**,
- n adet sınıf sarmalama işlemine tabi tutulursa **bileşen**,
- n adet bileşen sarmalama işlemine tabi tutulursa **kütüphane**,
- n adet kütüphane sarmalama işlemine tabi tutulursa **çerçeve yapılar** oluşur.

Böylece bize projemizde proje süresini ve maliyetini kısaltma üstünlüğünü verir.

Kalıtım

Öğrenci, öğretmen ve müdürün olduğu bir sistemi örnek olarak ele alalım.

Kalıtım Olmadan Sınıf Tasarımı

Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir.

```
// See https://aka.ms/new-console-template for more information
public class Öğrenci
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine("Ders Çalışıyorum.");
    }
}
```

Burada **Yaş özelliğinde** (**property**) hiçbir durum kontrolünün yapılmaması **{ get; set; }** olarak kodlanır. Benzer şekilde Öğretmen için ayrı bir sınıf;

```
// See https://aka.ms/new-console-template for more information
public class Öğretmen
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
    }
}
```

```

        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public int VediğiDersKodu { get; set; }
    public void dersVer()
    {
        Console.WriteLine("Ders Veriyorum.");
    }
}

```

Müdür için ayrı bir sınıf;

```

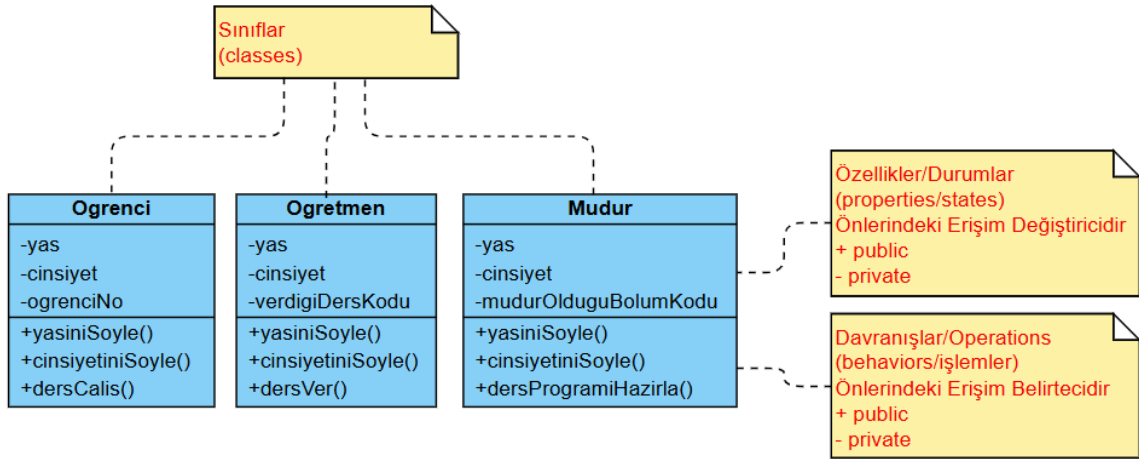
// See https://aka.ms/new-console-template for more information
public class Müdür
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public int MüdürOlduğuBölümKodu { get; set; }
    public void dersProgramıHazırla()
    {
        Console.WriteLine("Ders Programı Hazırlıyorum.");
    }
}

```

Bu sınıfların her birinde **Yaş** ve **Cinsiyet** gibi ortak **özellikler** (property) veya **yaşınıSöyle()** ve **cinsiyetiniSöyle()** gibi ortak **davranışlar** (behavior) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanırlar. Hâlbuki bu gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir.

Kalıtım İçeren Sınıf Uygulaması

Bütün bu ortak özellik ve davranışlar aşağıdaki diyagramdaki gibi tek bir sınıfta toplandığı zaman daha yönetilebilir bir kod elde edilir.



Şekil 23. Öğrenci, Öğretmen ve Müdür UML Sınıf Diyagramları

Yazılım analizi yapılırken takım içerisinde iletişimi kolaylaştırmak için aşağıdaki sınıf diyagramları (class diagram) kullanılır. Bunlar Unified Modelling Language-UML dilinin bir parçasıdır.

Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına kalıtım (inheritance) adı verilir. Yukarıdaki örnekte ortak özellik ve davranışlar **Kişi** sınıfına toplanarak aşağıdaki şekilde kod yeniden düzenlenebilir;

Aşağıda verilen ve Kişi olarak tanımlanan genel sınıf (general class), taban sınıf (base class) veya ebeveyn sınıf (parent class) olarak adlandırılır. Daha sonra tanımlanacak Öğrenci, Öğretmen ve Müdür sınıflarının ortak özellik ve davranışlarını barındırır.

```
// See https://aka.ms/new-console-template for more information
class Kişi
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}
```

Aşağıda taban sınıftan türemiş (derived) Öğrenci, Öğretmen ve Müdür sınıfları verilmiştir. Bu sınıflar genel sınıf olan Kişi sınıfının yanında kendine özel özellik ve davranışlara da sahiptir. Özel sınıf (private class), türemiş sınıf (derived class) veya çocuk sınıf (child class) olarak adlandırılan bu sınıflar, mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

```
// See https://aka.ms/new-console-template for more information
class Öğrenci : Kişi
{
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine("Ders Çalışıyorum.");
    }
}
```

```

class Öğretmen: Kişi
{
    public int VediğiDersKodu { get; set; }
    public void dersVer()
    {
        Console.WriteLine("Ders Veriyorum.");
    }
}
class Müdür: Kişi
{
    public int MüdürOlduğuBölümKodu { get; set; }
    public void dersProgramıHazırla()
    {
        Console.WriteLine("Ders Programı Hazırlıyorum.");
    }
}

```

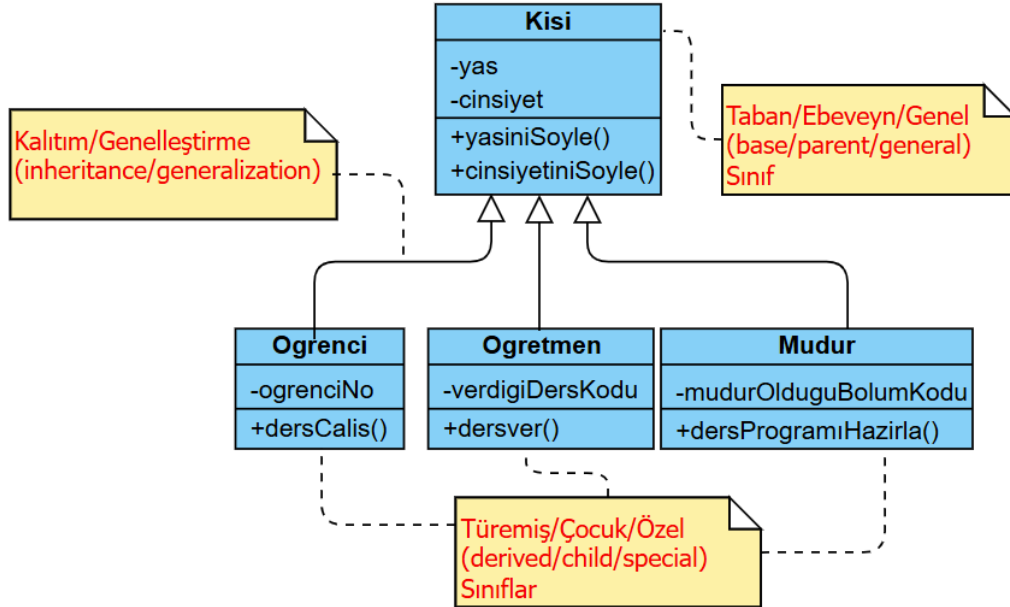
Programlamanın yapıldığı ana fonksiyonda bu üç sınıftan da nesne imal edilmiş ve çeşitli davranışlar göstermesi için kendilerine **ileti gönderilmiştir** (message-passing).

```

// See https://aka.ms/new-console-template for more information
Öğrenci ilhan = new Öğrenci();
Öğretmen mehmet = new Öğretmen();
Müdür abduallah = new Müdür();
ilhan.yaşınıSöyle();
ilhan.cinsiyetiniSöyle();
ilhan.dersÇalış();
mehmet.yaşınıSöyle();
mehmet.cinsiyetiniSöyle();
mehmet.dersVer();
abduallah.yaşınıSöyle();
abduallah.cinsiyetiniSöyle();
abduallah.dersProgramıHazırla();

```

Yeni haliyle sınıf diyagramları aşağıda verilmiştir;



Şekil 24. Kişi Taban Sınıfı Eklenmiş Kalıtım UML Sınıf Diyagramı

Değişim yönetimini kolayca yapma üstünlüğünü veren **kalıtımın** (inheritance) iki önemli özelliği vardır;

1. Paylaşılan bir **ortak kod** (shared-code) vardır.
2. Paylaşılan kodda yapılan **değişiklik anında alt sınıflara yansır** (snap-change).

Örneğimizden gidecek olursak, Kişi sınıfında yapılacak bir özellik veya davranış eklemesi anında bu sınıftan türemiş sınıflarda yapılmış olur.

Korumalı (**protected**) olarak taban sınıftan alınan durum ve davranışlara ait bir örnek aşağıda verilmiştir;

```
// See https://aka.ms/new-console-template for more information
Öğrenci damla = new Öğrenci();
damla.yaşınıSöyle(); // Kişi sınıfından miras alındı
damla.cinsiyetiniSöyle(); // Kişi sınıfından miras alındı
damla.dersÇalış(); // Öğrenci sınıfında tanımlandı
class Kişi
{
    protected uint Yaş { get; set; }
    protected char Cinsiyet { get; set; }
    private int TCKimlikNo { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public void kimliğiniSöyle()
    {
        Console.WriteLine($"Kimlik Numaram:{TCKimlikNo}");
    }
}
class Öğrenci : Kişi
{
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
        Console.WriteLine($"Cinsiyetim:{Cinsiyet}");
        //Console.WriteLine($"Kimlik Numaram: {TCKimlikNo}");
        /* Ebeveyn sınıfta TCKimlikNo private tanımlandığından erişilemez! */
        Console.WriteLine("Ders Çalışıyorum.");
    }
}
```

Türetilmiş bir sınıf, taban sınıfın yapıcıları, **yıkıcıları** (**destructor**) ve kopya yapıcıları dışında tüm taban sınıf yöntemlerini miras alır.

Nesne Yönelimli Programlamada sınıflar, birden fazla taban sınıftan miras alamayacak şekilde tasarlanır. Mantık olarak bir ev, birden fazla mimari plandan hazırlanmayacağından, **çoklu kalıtım** (**multiple inheritance**) sınıf tasarımında kullanılmaz! Bu nedenle saf nesne yönelimli diller olan Java ve .Net dillerinde **ara yüzler** (**interface**) ayrı bir anahtar kelimeyle tanımlandığından doğal olarak birden çok sınıftan miras alma engellenmiştir.

C# dilinde **kullanıcı tanımlı sınıflar** (**classes**) üstü kapalı olarak **object** sınıfından miras alır!

Miras Alınan Üyeleri Gizleme

Miras alınan bir **durum** (**state**), **özellik** (**property**) ve **davranışı** (**behavior**) gizlemek için **new** anahtar kelimesi kullanılır. Miras alınan bir üyeyi **new** ile gizlediğinizde, yeni üye taban sınıftaki yerini alır. Taban sınıfta tanımlanmış bir üye türemiş sınıfta görünür olduğu varsayılır. Çünkü taban sınıfta **private** veya bazı durumlarda **internal** olarak erişimi belirlenmiş üyeler, türemiş sınıfta zaten gizli olurdu. **new** değiştiricisini kullanmadan **public** veya **protected** üyeleri gizleyebilmenize rağmen bir derleyici uyarısı alırsınız. Bir üyeyi açıkça gizlemek için **new** kullanırsanız, bu uyarıyı bastırır.

```
// See https://aka.ms/new-console-template for more information
Taban taban = new Taban(10);
Türemiş türemiş = new Türemiş(30);
taban.Yöntem(); // Taban Sınıfıta 10
türemiş.Yöntem(); /* Türemiş Sınıfıta 0: Çünkü;
                    türemiş sınıfıta tanımlanan alan değişkeni
                    başlangıç değeri sıfırdır ve taban sınıfıdaki
                    x gizlenmiştir. */

public class Taban
{
    public Taban(int x)
    {
        this.x = x;
    }
    public int x;
    public void Yöntem()
    {
        Console.WriteLine($"Taban Sınıfıta {x}");
    }
}

public class Türemiş : Taban
{
    public Türemiş(int x) : base(x) { }
    new public int x;
    new public void Yöntem()
    {
        Console.WriteLine($"Türemiş Sınıfıta {x}");
    }
}
```

Aşırı Yükleme

Aşırı yükleme (**overloading**); aynı davranışın farklı **yöntemlerle** (**method**) yeniden tanımlanmasıdır. Yöntemler, yapıcılar ve işleçlerin bazıları aşırı yüklenebilir;

Yöntemlerin Aşırı Yüklmesi

Kısaca yöntemlerin aşırı yüklemesi, farklı parametrelerle yöntemin yeniden tanımlanmasıdır. Aşırı yüklemede yöntemin kimliği değişmez. Farklı parametrelerle dönüş tipi de farklı olarak tanımlanabilir. Aşağıda bir fonksiyonun aşırı yüklemesine bir örnek verilmiştir.

```
// See https://aka.ms/new-console-template for more information
Hesap h = new Hesap();
Console.WriteLine(h.topla(10, 20)); // 30
Console.WriteLine(h.topla(1, 2.0)); // 3
Console.WriteLine(h.topla(3.0, 1)); // 4
Console.WriteLine(h.topla(3.0, 3.0)); // 6

class Hesap
{
    public int topla(int p1, int p2)
    // "topla" kimlikli yöntem
    {
        return p1 + p2;
    }
    public double topla(double p1, int p2)
    // aşırı yüklenmiş "topla" kimlikli yöntem
    {
        return p1 + p2;
    }
    public double topla(int p1, double p2)
    // aşırı yüklenmiş "topla" kimlikli yöntem
    {
```

```

        return p1 + p2;
    }
    public double toplad(double p1, double p2)
    // aşırı yüklenmiş "topla" kimlikli yöntem
    {
        return p1 + p2;
    }
}

```

Farklı parametrelerle dönüş tipi de farklı olarak tanımlanabilir;

```

public int toplad(int p1, int p2);
public string toplad(int p1, int p2); //Derlenmez! Parametreler farklı olmalı!

```

Yöntemler varsayılan argümanlar ile tanımlandığında aşırı yükleme tanımlanmasında varsayılan argümanlar ile çakışmamasına çok dikkat edilmelidir. Aşağıdaki örnekte varsayılan argüman olan toplad değil diğeri ile işlem yapılır!

```

// See https://aka.ms/new-console-template for more information
Hesap h = new Hesap();
Console.WriteLine(h.toplad(10)); //13

class Hesap
{
    public int toplad(int p1, int p2=1) // "toplad" kimlikli yöntem
    {
        return p1 + p2;
    }
    public int toplad(int p1) // "toplad" kimlikli aşırı yüklenmiş yöntem
    {
        return p1 + 3;
    }
}

```

Yapıcıların Aşırı Yüklenmesi

Yapıcıların görevi kısaca imal edilecek nesnelerin durumlarına (state) ilk değer vermektir. Nesneleri de dışarıdan verilen parametrelerle farklı durumlarda imal etmek için aşırı yüklenmiş yapıcıları kullanabiliriz. Yani farklı parametrelerle yapıcıları yeniden tanımlayabiliriz. Yapıcıların kimliği sınıf kimliğiyle aynıdır. Aşırı yüklemede tanımlanacak yapıcının kimliği de sınıf kimliği ile aynı olur.

Aşağıda aşırı yüklenmiş yapıcı, parametrelili (parameterized constructor),

```

// See https://aka.ms/new-console-template for more information
Kompleks c1 = new Kompleks();
c1.Yaz();// 0 + 0i
Kompleks c2 = new Kompleks(5, 2);
c2.Yaz();// 5 + 2i
Kompleks c3 = new Kompleks(8);
c3.Yaz();// 8 + 0i
Kompleks c4 = c2; // c4 için Kopya Yapıcı Çalışır
c4.Yaz();// 5 + 2i

public class Kompleks
{
    private int gercek;
    private int sanal;
    public Kompleks() // Ön tanımlı (default) yapıcı
    {
        sanal = 0;
        gercek = 0;
    }
    public Kompleks(int r = 0, int i = 0) // Aşırı yüklenmiş parametrelili yapıcı
    {

```

```

        gercek = r;
        sanal = i;
    }
    public Kompleks(int r = 0) // Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        gercek = r;
        sanal = 0;
    }
    public Kompleks(Kompleks p) //Kopya Yapıcı: Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        gercek = p.gercek;
        sanal = p.sanal;
    }
    public void Yaz()
    {
        Console.WriteLine($"{gercek} + {sanal}i");
    }
}

```

Yukarıdaki örnekte;

- Aynı sınıftan bir nesneyi bir başka imal edilmiş nesnenin durumlarına sahip olarak imal edebiliriz. Bir nesneyi bir yapıcıya argüman olarak geçirmek ve yeni nesneye argümanın durumlarını vermek gerekir. **Kompleks(Kompleks p)** şeklinde kodlanmış bu yapıcıya **kopya yapıcı** (**copy constructor**) adı verilir.
- Yukarıdaki örnekte **Kompleks(int r = 0, int i = 0)** ve **Kompleks(int r = 0)** yapıcıları **parametrelili yapıcı** (**parameterized constructor**) olarak adlandırılır.

Taban Sınıfın Yapıcısının Çağırılması

Kalıtım (**inheritance**) durumunda ise taban sınıfın yapıcısı çağrılır. Bu duruma aşağıdaki örnek verilebilir;

```

// See https://aka.ms/new-console-template for more information
Öğrenci damla = new Öğrenci(1234, 25, 'K');
damla.yaşınıSöyle(); //Yaşım: 25
damla.cinsiyetiniSöyle(); //Cinsiyetim: KADIN
damla.kimliğiniSöyle(); //Kimlik Numaram: 0
damla.dersÇalış(); //Ders Çalışıyorum.
class Kişi
{
    private uint Yaş { get; set; }
    private char Cinsiyet { get; set; }
    private int TCKimlikNo { get; set; }
    public Kişi(uint yaş, char cinsiyet, int tckimlikno)
    {
        this.Yaş = yaş;
        this.Cinsiyet = cinsiyet;
        this.TCKimlikNo = tckimlikno;
    }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}

```

```

    public void kimliğiniSöyle()
    {
        Console.WriteLine($"Kimlik Numaram:{TCKimlikNo}");
    }
}
class Öğrenci : Kişi
{
    public Öğrenci(int öğrencino,
        uint yaş = 18,
        char cinsiyet = 'E',
        int tckimlikno = 0) : base(yaş, cinsiyet, tckimlikno)
    {
        this.ÖğrenciNo = öğrencino;
    }
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine("Ders Çalışıyorum.");
    }
}

```

Yukarıdaki örnekte olduğu gibi **ön tanımlı argümanlar** (default argument) da yapıcılarda kullanılabilir. Yapıcıların aşırı yüklenmesinin çeşitli üstünlükleri vardır;

- Nesne imal etmede esneklik: Aynı sınıftan farklı durumlara sahip nesneler imal edilebilir.
- Gelişmiş kod bakımı ile daha temiz ve okunabilir kod oluşur.
- Nesne imal etme mantığı diğer nesnelerden/programlardan gizlenir.
- Kopya yapıcılar ile nesne klonlama basitleşir.

İşleçlerin Aşırı Yüklenmesi

Hali hazırda tanımlı olan tekli (unary), aritmetik, eşitlik ve karşılaştırma **işleçlere** (operator) de bizim tanımladığımız sınıflardan imal edilen nesneler üzerinde işlem yapmak için aşırı yükleme yapılabilir.

İşleçler	Açıklama
<code>+x, -x, !x, ~x, ++, --, true, false</code>	true ve false işleçleri birlikte aşırı yüklenmelidir
<code>x + y, x - y, x * y, x / y, x % y, x & y, x y, x ^ y, x << y, x >> y, x >>> y</code>	
<code>x == y, x != y, x < y, x > y, x <= y, x >= y</code>	== ile != ve < ile > ve <= ile >= birlikte aşırı yüklenmelidir.

Tablo 20. Aşırı Yüklenebilecek İşleçler

Aşağıda işleç aşırı yüklemesini göstermek amacıyla yeni bir **Kompleks** sınıfı tanımlanmış ve üzerinde toplama işleci aşırı yüklenmiştir;

```

// See https://aka.ms/new-console-template for more information
Kompleks c1 = new Kompleks(3, 7);
c1.Yaz();// 3 + 7i
Kompleks c2 = new Kompleks(5, 2);
c2.Yaz();// 5 + 2i
Kompleks c3 = c1 + c2;
c3.Yaz();// 8 + 9i

public class Kompleks
{
    private int gercek;
    private int sanal;
    public Kompleks(int r = 0, int i = 0)
    {
        gercek = r;
    }
}

```

```

        sanal = i;
    }
    public static Kompleks operator +(Kompleks c1, Kompleks c2)
    {
        Kompleks temp = new Kompleks();
        temp.gercek = c1.gercek + c2.gercek;
        temp.sanal = c1.sanal + c2.sanal;
        return temp;
    }
    public void Yaz()
    {
        Console.WriteLine($"{gercek} + {sanal}i");
    }
}

```

this Göstericisi

İmlal edilen her nesne, **this** adı verilen önemli bir gösterici aracılığıyla kendi adresine erişebilir. **this göstericisi** (**this pointer**) tüm üye yöntemler için örtük bir parametredir. Aşağıda karmaşık sayıları örneği verilmiştir;

```

// See https://aka.ms/new-console-template for more information
Kompleks c1 = new Kompleks();
c1.Yaz();// 0 + 0i
Kompleks c2 = new Kompleks(5, 2);
c2.Yaz(); //5 + 2i
Kompleks c3 = new Kompleks(8);
c3.Yaz(); //8 + 0i
Kompleks c4 = c2; // c4 için Kopya Yapıcı Çalışır
c4.Yaz(); //5 + 2i

public class Kompleks
{
    private int gercek;
    private int sanal;
    public Kompleks()
    // Ön tanımlı (default) yapıcı
    {
        sanal = 0;
        gercek = 0;
    }
    public Kompleks(int r = 0, int i = 0) // Aşırı yüklenmiş parametrelili yapıcı
    {
        this.gercek = r;
        this.sanal = i;
    }
    public Kompleks(int r = 0) // Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        this.gercek = r;
        this.sanal = 0;
    }
    public Kompleks(Kompleks p) //Kopya Yapıcı:Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        this.gercek = p.gercek;
        this.sanal = p.sanal;
    }
    public void Yaz()
    {
        Console.WriteLine($"{gercek} + {sanal}i");
    }
}

```

Statik Üyeler ve Statik Sınıf

Programda çalışmaya başladığı anda bazı değişkenler ve yöntemlere henüz bellekte tahsis edilmemiş veya istenilen nesneler henüz imal edilmemiştir. Program daha çalışmadan bu tip değişken, yöntem ve nesnelerin bellekte hazır olması istenir. Uygulamamızın daha belleğe yüklenirken ve yaşamı boyunca bellekte bulunacak değişken, yöntem ve nesneler **static** olarak tanımlanır.

Programlarda sıkça kodladığımız **static void Main(string[] args)** fonksiyonu da bu nedenle **static** olarak tanımlanır. **Main** yöntemi dışında da bazı nesnelerin imal edilmesi veya değişken ve yöntemlerin belleğe program belleğe yüklenirken yüklenmesi gerekir. İşte bu değişkenler ve yöntemler de statik olarak tanımlanır. Statik sınıflar;

- Doğası gereği nesne yapıcısı barındırmazlar!
- Bu sınıflar türetilemez!
- Sınıfın tüm üyeleri statik olmak zorundadır!

Statik olan değişken veya yöntemler yani **statik üyeler** (**static member**), **örnek üyeleri** (**instance member**) gibi nesne ile imal edilip nesne ile ölmediklerinden, nesnelere ait değil sınıfa ait olarak nitelendirilir. Yani program bellekte kaldığı sürece statik sınıf ve üyeleri hayattadır.

```
// See https://aka.ms/new-console-template for more information
string bağlantı = Yardımcı1.GetConnection();
Yardımcı1.GenelSayaç++;
ulong f = Yardımcı1.Faktöriyel(5);

static class Yardımcı1 /* Statik sınıf */
{
    public static ulong GenelSayaç = 0;
    public static string GetConnection()
    {
        return "Data Source=(LocalDB)\\MSSQLLocalDB;" +
            "Integrated Security=True;Connect Timeout=30;Encrypt=True";
    }
    public static ulong Faktöriyel(uint n)
    {
        if (n > 1) return n * Faktöriyel(n - 1);
        else return 1;
    }
}
```

Sabit Alanlar

Alanların (**field**) nesnelere ilişkin **durumları** (**state**) yani verileri tuttuğu anlatılmıştı. Sabitler, derleme zamanında bilinen ve programın ömrü boyunca değişmeyen değişmez değerlerdir. Sabit alan ve yerel değişkenler **const** sıfatıyla tanımlanır.

Metinler (**string**) dışındaki referans tipleri sabitlere yalnızca **null** olarak ilk değer verilebilir! **Sınıflar** (**class**), **yapılar** (**struct**) ve **diziler** (**array**) dahil olmak üzere kullanıcı tanımlı tipler **const** tanımlanamaz! Çalışma zamanında yapıcı ile bir kez imal edilen ve bundan sonra değiştirilemeyen bir nesne, yapı veya dizi oluşturmak için **readonly** sıfatı kullanılır.

```
// See https://aka.ms/new-console-template for more information
const float PI = 3.1415f;
Takvim t = new Takvim();
t.Yaz();
/*
Ay Sayısı:12
Saat Sayısı:8786 */
Console.WriteLine(Takvim.TakvimAdı);
//const alanlar static olarak algılanır ve Sınıf İsmi ile erişilir.
Console.WriteLine(t.günler[0]); //Pazartesi
//readonly alanlar ise static olarak algılanmaz ve nesne üzerinden erişilir.
```

```
class Takvim
{
    public const int AySayısı = 12;
    public const int GünSayısı = 365;
    public const int HaftaSayısı = 52;
    public const int SaatSayısı = GünSayısı * 24 + 6;
    public const string TakvimAdı = "Takvim";
    public readonly string[] günler = { "Pazartesi", "Salı", "Çarşamba",
                                         "Perşembe", "Cuma", "Cumartesi",
                                         "Pazar" };

    public void Yaz()
    {
        Console.WriteLine($"Ay Sayısı:{AySayısı}");
        Console.WriteLine($"Saat Sayısı:{SaatSayısı}");
    }
}
```

readonly Kelimesi

readonly anahtar kelimesi aşağıdaki bağlamda kullanılabilen bir sıfattır.

1. Salt okunur bir **alan** (field) tanımlanmasında; alana atama yalnızca bildirimde veya aynı sınıftaki bir yapıcıda gerçekleşir.

```
class Takvim
{
    public readonly int AySayısı=12;
    public readonly int HaftaSayısı;
    public Takvim()
    {
        HaftaSayısı = 365 / 7;
        /* Sınıf içinde bile olsa Başka yerde bu alanlar değiştirilemez. */
    }
}
```

2. Salt okunur bir **yapı** (struct) tipi tanımında; yapı tipinin değiştirilemez olduğunu belirtir.

```
// See https://aka.ms/new-console-template for more information
var p1 = new Koordinat(0, 0);
Console.WriteLine(p1); // (0, 0)
var p2 = p1 with { X = 3 };
Console.WriteLine(p2); // (3, 0)
var p3 = p1 with { X = 1, Y = 4 };
Console.WriteLine(p3); // (1, 4)

public readonly struct Koordinat
{
    public Koordinat(double x, double y)
    {
        X = x;
        Y = y;
    }
    public double X { get; init; }
    public double Y { get; init; }
    public override string ToString()
    {
        return $"({X}, {Y})";
    }
}
```

3. **Örnekleme** (instantiation) yapılarak yani nesne imal edilerek salt okunur bir üye nesne tanımlanıyorsa; bu örnek üyesinin durumunu değiştirmedini belirtir.

```
// See https://aka.ms/new-console-template for more information
Kişi ali = new Kişi("Ali", "YILMAZ");
ali.Yaz();

class Kitap
{
    public string Adı { get; set; }
    public Kitap(string adı)
    {
        this.Adı = adı;
    }
}

class Kişi
{
    public string Adı { get; set; }
    public string Soyadı { get; set; }
    public readonly Kitap matematikKitabı = new Kitap("Calculus I");
    public readonly Kitap kitap;
    public Kişi(string adı, string soyadı)
    {
        this.Adı = adı;
        this.Soyadı = soyadı;
        kitap = new Kitap("C# Programlama");
    }
    public void Yaz()
    {
        Console.WriteLine($"Adı:{Adı}, Soyadı:{Soyadı}");
        Console.WriteLine($"Matematik: {matematikKitabı.Adı}");
        Console.WriteLine($"Kitap: {kitap.Adı}");
    }
}
```

4. Bir `ref readonly` metodu dönüşünde, `readonly` kelimesi metodun bir referans döndürdüğünü ve bu referansa yazmalara izin verilmediğini belirtir.

Değiştirilemez Nesneler

Nesne imal edildikten sonra durumları değiştirilebilen nesneler **değiştirilebilir** (*mutable*) nesnelerdir. Bir nesnenin durumlarının değiştirilebilir olması, dahili verilerinin değiştirilebileceği ve bu da nesnenin bütünlüğünün korunmasında olası yan etkilere ve zorluklara yol açabileceği anlamına gelir. Değiştirilebilir sınıfların yaygın örnekleri arasında, elemanlarının eklenebildiği, kaldırılabilirdiği veya değiştirilebildiği birçok koleksiyon türü bulunur.

Değiştirilebilir nesneler yerine değiştirilemez nesneler kullanmak istediğimiz durumlar vardır. Değiştirilemez nesneler **iş parçacığı güvenli** (*thread safe*), ancak diğer yandan birçok durumda performans sorunlarına neden olabilirler. Her geliştiricinin günlük olarak kullandığı en popüler değiştirilemez nesne **dizgilerdir** (*string*). Metinlerle yapılan her işlem sonrasında yeni bir metin ortaya çıkar.

Değiştirilemez bir nesne elde etmek için sınıflardaki alanlar `readonly` tanımlanır. Bu sınıfın yapıcısında ilke değerler verilir ve nesne imalatı olduktan sonra bu alanlar değiştirilemez.

```
// See https://aka.ms/new-console-template for more information
Takvim t = new Takvim();
t.Yaz();
Console.WriteLine($"Saat Sayısı:{t.SaatSayısı}");
Console.WriteLine(t.TakvimAdı.ToUpper());
/* TakvimAdı immutable string sınıfındandır. Burada değeri "Takvim" aynen kalır ve
   Büyük harfe çevrilmiş olan yeni bir string imal edilir. */

class Takvim
{

```

```

public readonly int AySayısı;
public readonly int GünSayısı;
public readonly int HaftaSayısı;
public readonly int SaatSayısı;
public readonly string TakvimAdı;
public Takvim()
{
    AySayısı = 12;
    GünSayısı = 365;
    HaftaSayısı = GünSayısı / 7;
    SaatSayısı = GünSayısı * 24 + 6;
    TakvimAdı = "Takvim";
}
public void Yaz()
{
    Console.WriteLine($"Hafta Sayısı:{HaftaSayısı}");
    Console.WriteLine($"Gün Sayısı:{GünSayısı}");
}
}

```

Geçersiz Kılma

Yöntemi **geçersiz kılma** (**overriding**), nesne yönelimli programlamanın, taban sınıfta önceden tanımlanmış bir yöntemi türemiş bir sınıfta yeniden tanımlamasına olanak tanıyan bir kavramdır. Yani taban sınıfın davranışının türemiş sınıfta değiştirilmesidir. Geçersiz kılınacak yöntem, taban sınıfta ya **sanal** (**virtual**) ya da **soyut** (**abstract**) olarak tanımlanmalıdır.

```

class Taban-Sınıf-Kimliği {
    // geçersiz kılınacak yöntem:
    erişim-belirleyici virtual|abstract geri-dönüş-veri-tipi yöntem-kimliği() {
    }
};

class Türemiş-Sınıf-Kimliği: Taban-Sınıf-Kimliği {
    // geçersiz kılan yöntem:
    erişim-belirleyici override geri-dönüş-veri-tipi yöntem-kimliği() {
    }
};

```

Her iki sınıf ta da yöntemin kimliği aynıdır. Taban sınıftan imal edilen nesne taban sınıfın yöntemini, türemiş sınıftan imal edilen nesne ise türemiş sınıfın yöntemini icra eder.

```

// See https://aka.ms/new-console-template for more information
Baba baba = new Baba();
Çocuk çocuk = new Çocuk();
baba.yap(); //Baba bu şekilde yapar ...
çocuk.yap(); //Çocuk başka şekilde yapar..

class Baba
{
    public virtual void yap()
    {
        Console.WriteLine("Baba bu şekilde yapar ...");
    }
}
class Çocuk : Baba
{
    public override void yap()
    {
        Console.WriteLine("Çocuk başka şekilde yapar...");
    }
}

```

Ara Yüzler veya Roller

Ara yüzler (**interface**) nesnelerin sahip oldukları **rollerdir** (**role**). Ancak saf nesne yönelimli dillerde **ara yüzler** **interface** saklı kelimesiyle tanımlanırlar.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav/değerlendirme yapma, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Ara yüzler de referans tiplerdir. Bir sınıfa, bir veya birden fazla **ara yüz uygulanabilir** (**interface implementation**). Buna **ara yüz gerçekleştirme** (**interface realization**) adı da verilir. Ara yüzler içinde yöntemlere ilişkin **desenler** (**signature**) yazılır. Yani sadece yöntem başlığı yazılır. Gövdesi yazılmaz. Ayrıca yöntemler için erişim belirleyici kullanılmaz.

```
// See https://aka.ms/new-console-template for more information
Kişi ilhan = new Kişi("İlhan", "ÖZKAN");
//imal edilen nesne tüm arayüzdeki davranışları gösterebilir:
ilhan.AdınıSöyle(); // İlhan ÖZKAN
ilhan.Öğret(); // Öğretiyorum...
ilhan.GitarÇal(); // Gitar Çalıyorum...
ilhan.HanımaYardımEt(); // Hanıma Yardım Ediyorum...

//imal edilen nesne sadece bir roldeki davranışları da gösterebilir:
IBaba baba = ilhan;
baba.ÇocuğaBak(); // Çocuğa Bakıyorum...
baba.HanımaYardımEt(); // Hanıma Yardım Ediyorum...
İMüzisyen müzisyen = ilhan;
müzisyen.GitarÇal(); // Gitar Çalıyorum...
müzisyen.PiyanoÇal(); // Piyano Çalıyorum...
İÖğretmen öğretmen = ilhan;
öğretmen.DersNotuHazırla(); // Ders notu hazırlıyorum...
öğretmen.Öğret(); // Öğretiyorum...

interface IBaba //Baba Rolündeki Davranışlar:
{
    void ÇocuğaBak(); // yöntemin gövdesi olmaz!
    void HanımaYardımEt();
}
interface İMüzisyen //Müzisyen Rolündeki Davranışlar:
{
    void GitarÇal();
    void PiyanoÇal();
}
interface İÖğretmen//Öğretmen Davranışları:
{
    void Öğret();
    void DersNotuHazırla();
}
class Kişi : IBaba, İMüzisyen, İÖğretmen
{
    /*
     * Arayüz Gerçekleştirme/Uygulama(interface realization/implementation):
     * Gerçekleştirilen IBaba, İMuzisyen ve İOgretmen arayüzlerinde bulunan yöntemler
     * Kişi sınıfında hepsi gövdeleriyle tanımlanmalıdır!
     */
    public string Adı { get; set; }
    public string Soyadı { get; set; }
    public Kişi(string adı, string soyadı)
    {
        this.Adı = adı;
    }
}
```

```

        this.Soyadı = soyadı;
    }
    public void AdınıSöyle()
    {
        Console.WriteLine($"{Adı} {Soyadı}");
    }
    public void ÇocuğaBak()
    {
        Console.WriteLine("Çocuğa Bakıyorum..");
    }
    public void HanımaYardımEt()
    {
        Console.WriteLine("Hanıma Yardım Ediyorum..");
    }
    public void PiyanoÇal()
    {
        Console.WriteLine("Piyano Çalıyorum..");
    }
    public void GitarÇal()
    {
        Console.WriteLine("Gitar Çalıyorum..");
    }
    public void Öğret()
    {
        Console.WriteLine("Öğretiyorum..");
    }
    public void DersNotuHazırla()
    {
        Console.WriteLine("Desr Notu Hazırlıyorum..");
    }
}

```

Nesne Yıkıcı

Sonlandırıcılar (**finalizer**) olarak adlandırılan **yıkıcı** (**destructor**), bir nesne yok edileceği zaman otomatik olarak çağrılan yapıcı gibi bir yöntemdir. Bir sınıf **örneği** (**instance**) **çöp toplayıcı** (**garbage collector-GC**) tarafından toplanırken gerekli olan son temizliği gerçekleştirmek için kullanılır. Bir sınıf içinde yalnızca bir yıkıcı tanımlanabilir. Yıkıcı, bir nesne çöp toplayıcı tarafından bellekten silinmeden hemen önce çalışan özel bir yöntemdir. Aşağıdaki gibi sınıf kimliği önünde yer alan **dalga** (**tilda**) karakteri ile tanımlanır. Parametre alamaz ve **erişim belirleyici** (**access modifier**) kullanılamaz. Manuel olarak çağrılmaz, sadece çöp toplayıcı tarafından otomatik çağrılır ve **static** olamaz.

```

~sınıf-kimliği() {
    // ...
}

```

Nesnelerin üstü kapalı olarak **object** sınıfından miras aldığı söylenmişti. Yıkıcı, nesnenin taban sınıfında **Finalize** yöntemini dolaylı olarak çağırır. Bu nedenle, bir sonlandırıcıya yapılan çağrı dolaylı olarak aşağıdaki koda çevrilir:

```

protected override void Finalize()
{
    try
    {
        // Cleanup statements...
    }
    finally
    {
        base.Finalize();
    }
}

```

Mutlaka yıkıcı bir yöntem çağırmak gerekiyorsa kaynak yönetimi için en iyi yöntem **IDisposable** ara yüzü gerçekleştirmek ve bu ara yüzün sunduğu **Dispose()** metodunu kullanmaktır.

```
class MyClass : IDisposable
{
    private bool disposed = false;

    // Dispose metodu
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this); // Sonlandırıcının tekrar çalışmasını engeller
    }

    protected virtual void Dispose(bool disposing)
    {
        if (!disposed)
        {
            if (disposing)
            {
                // Yönetilen kaynakları temizle
            }
            // Yönetilmeyen kaynakları temizle
            disposed = true;
        }
    }

    // Destructor (finalizer): Sonlandırıcı
    ~MyClass()
    {
        Dispose(false);
    }
}
```

Bu yapı, hem **Dispose()** çağrıldığında, hem de çağrılmazsa ve çöp toplayıcı devreye girerse temizliği garanti altına alır. **GC.SuppressFinalize(this)** ifadesi, eğer **Dispose()** çağrıldıysa yıkıcının çalışmasını engeller, çünkü temizlik zaten yapılmıştır.

Çok Biçimlilik

Çok biçimlilik (**polymorphism**), birden çok nesnenin olduğu bir ortamda, verilen **aynı iletiye** (**message**) karşı, nesnelerin **farklı davranış** (**behavior**) göstermeleridir (**same message-different behavior**).

Çok biçimliliğin uygulanabilmesi (**implementation**) için üç şart vardır;

- **Kalıtım** (**inheritance**): Ortak davranışın tanımlandığı taban sınıf ve bu davranışın değiştiği türeyen sınıflar.
- Nesnelere aynı iletiyi verebilmek için ortak davranışı tanımlayan **Sanal Yöntem** (**virtual method**).
- **Geçersiz Kılan Yöntem** (**override method**): Devralınan davranışı kendine uyarlayan ve devralınan yöntemi geçersiz kılan yeni bir yöntem.

```
// See https://aka.ms/new-console-template for more information

SoyutKişi[] kişiler = new Kişi[3];
kişiler[0] = new Müdür("İlhan");
kişiler[1] = new Öğrenci("Mehmet");
kişiler[2] = new Öğretmen("Abdullah");
foreach (Kişi kişi in kişiler)
    kişi.raporYaz();
/*
Müdür İlhan Rapor Yazıyor..
Öğrenci Mehmet Rapor Yazıyor..
Öğretmen Abdullah Rapor Yazıyor..
*/
```

ÇOKBİÇİMLİLİK=AYNI MESAJ KARŞISINDA FARKLI DAVRANIŞ

(PolyMorphism: Same Message -> Different Behaviour):

Bütün nesnelere aynı ileti gönderiliyor ama
hepsi farklı davranış gösteriyor.

*/

```
abstract class SoyutKişi //Soyut Kisi Sınıfı
{
    public abstract void raporYaz();
    public string Adı { get; set; }
    public SoyutKişi(string adı)
    {
        Adı = adı;
    }
}
class Kişi : SoyutKişi
{
    public Kişi(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Kişi {Adı} Rapor Yazıyor..");
    }
}
class Öğrenci : Kişi
{
    public Öğrenci(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Öğrenci {Adı} Rapor Yazıyor..");
    }
}
class Öğretmen : Kişi
{
    public Öğretmen(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Öğretmen {Adı} Rapor Yazıyor..");
    }
}
class Müdür : Kişi
{
    public Müdür(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Müdür {Adı} Rapor Yazıyor..");
    }
}
```

Çok biçimlilik bize **çalıştırma anında** (run time) üstünlük sağlar. Bu nedenle nesneler de **çalıştırma anında** imal edilmiştir. Program çalıştırıldığında her **kişi** nesnesine aynı ileti verilmiş ama bu nesneler farklı davranış göstermiştir.

Yazılımcı için çok biçimlilik, farklı tipte nesnelerin aynı isimli **yöntemleri** (method) çağrıldığında her bir nesnenin kendine özgü davranışı gerçekleştirme yeteneği olup **çalıştırma anında** üstünlük sağlar. Yani yazılımcı, nesnenin tipine bakmaksızın, nesnelerden aynı davranışı göstermesini ister. Burada gösterilecek davranışın, taban sınıfın davranışı mı olduğu veya türeyen sınıfın davranışı mı olduğuna **çalıştırma anında** (run time) karar verilir. İşte bu karar verme işlemine **geç bağlama** (late binding) veya **dinamik bağlama** (dynamic binding) adı verilir.

Esnekliğin istenmediği bazı durumlarda ise karar verme işlemi derleme anında yapılır. Buna ise **statik bağlama** (static binding) adı verilir. İkisi arasındaki seçim programcı tarafından yapılır.

Yazılım yapılacak sistemin başlangıç koşullarına bağlı tasarım kararlarının tümü bilinemez. Ayrıca sistemin genişlemesi için gerekli tüm kodlamanın bitirilmesi beklenemez. İşte bunu gerçekleştiren dinamik bağlama, yazılım mimarisinin daha esnek (mevcut bileşenin sistemde yeniden yapılandırmasının kolayca yapılması) ve genişleyebilir (yeni bileşenlerin kolayca sisteme eklenmesi) olmasını sağlar.

Sınıf Çeşitleri

Soyut Sınıf

Soyut sınıfların (**abstract class**) bir **örneği** (**instance**) olamaz. Yani bu sınıftan nesne imal edilemez. Bu sınıfın en az bir **soyut yöntemi** (**abstract method**) olur. Soyut sınıflar hangi davranışın gösterileceğini belirler ama bu davranışların nasıl gösterileceğini anlatmaz. Bu nedenle soyut yöntem için gövde tanımlanmaz.

```
public abstract class Soyut
/* soyut (abstract) sınıf hangi davranışlara ve özelliklere sahip olacağını söyler. */
{
    public abstract void Yöntem();
    //Soyut Yöntemler için gövde tanımlanmaz!
}
```

Somut Sınıf

Somut sınıflardan (**concrete class**) nesne imal edilebilir yani **örneği** (**instance**) olabilir. Bu sınıflarda davranış ve durumlar eksiksiz tanımlıdır. Bir sınıf tanımlarken soyut bir sınıfa genel davranış ve durumları toplamak ve ondan miras alarak somut sınıfları tanımlamak her zaman iyi bir seçimdir.

```
public class Somut : Soyut
/* Somut (concrete) sınıf, her zaman soyut sınıftan miras almak zorunda değildir. */
{
    public override void Yöntem()
    /* soyut yöntem, soyut olan ve miras alınan Yöntem" metodunu * geçersiz kılar (override)
       Ve mutlaka ete kemiğe bürünen bir davranışı tanımlamak zorundadır. */
    {
        /* soyut olarak gösterilecek davranış burada kodlanır. */
    }
}
```

Taban Sınıf

Taban sınıf (**base class**), miras alınan sınıf veya genel özellik ve davranışların toplandığı yani genelleştirmenin yapıldığı genel sınıf (**general class**) veya ebeveyn (**parent**) sınıftır.

```
public class TabanSınıf
{
    public virtual void Yöntem()
    {
    }
}
```

Türetilmiş Sınıf

Türetilmiş sınıf (**derived class**), miras alan sınıf veya davranış ve durumlar hakkında uzmanlaşmış yani uzman (**special class**) veya çocuk (**child class**) sınıftır. Türetilmiş ya da miras almış sınıf.

```
public class TüretilenSınıf : TabanSınıf
{
    public override void Yöntem()
    {
    }
}
```

Kök Sınıf

Kök sınıf (**root class**), babası olmayan ya da yetim sınıf olup aynı zamanda baba sınıftır.

Yaprak Sınıf

Yaprak sınıf (**leaf class**), kendisinden henüz miras alan bir sınıf olmayan bir sınıftır. Kısaca türememiş sınıftır.

Kısır Sınıf

Kısır sınıf (**sealed class**) kendisinden türeyen bir sınıf tanımlayamayan bir sınıftır. Kısır yöntem (**sealed method**) ise türeyen sınıfta **geçersiz kılınamaz** (**override**) yöntemidir.

```
public class Sınıf
{
    public virtual sealed void Yöntem()
    { // Kısır bir Yöntem tanımlanıyor
    }
}
public class Çocuk : Sınıf
{
    public override void Yöntem() /* Derleme Sırasında Hata:
    Kısır yöntem geçersiz kılınamaz (override edilemez) */
    {
    }
}
public sealed class Kısır //Kısır (sealed) sınıf: miras bırakmaz.
{
    public Kısır() { }
    public void Yöntem()
    {
    }
}
public class Türemiş : Kısır
/* Derleme Sırasında Hata: Kısır sınıftan miras alınamaz. */
{
}
```

Tekil Sınıf

Tekil sınıftan (**singleton class**) yalnızca bir nesne imal edilebilir. Yani yalnızca bir örneği vardır. Bunun olabilmesi için **mahrem** (**private**) yapıcısı olması gerekir. *Tekil Nesne* başlığında incelenecektir.

Yardımcı Sınıf

Yardımcı sınıf (**utility class**), üyeleri statik olan sınıflardır. Ayrıca evrensel **yapılandırma ayarlarını** (**configuration settings**) veya **değişmezleri** (**literal**) depolamak için kullanılabilir. Bir kaynak havuzunu (önbellek, veri tabanı bağlantı havuzu, vb.) yönetmek ve örnekler arasında paylaşılan bir günlük sistemi uygulamak için yararlıdır. Bunların dışında, **izleme yöntemi** (**trace method**) çağrılar için de kullanılabilir. Yardımcı sınıflar **statik** (**static**) sınıflardır ve bu sınıflardan nesne imal edilemez. Yani Yardımcıları bulunmaz.

Nesnelerle İlgili İşleçler

Tip Kontrol İşleci

is işleci bir nesnenin bir sınıfa ait olup olmadığını kontrol eder.

```
object obj = "Hello, World!";
if (obj is string)
{
    Console.WriteLine("obj nesnesinin veri tipi stringdir."); // Bunu konsola yazar
}
```

Güvenli Tip Dönüşüm İşleci

as işleci, istisna bir duruma düşmeden güvenli tip dönüşümünü (safe type casting) gerçekleştirir.

```
object obj = "Merhaba";
string str = obj as string;
if (str != null)
{
    Console.WriteLine("Tip dönüşümü başarılı: " + str); // Bunu konsola yazar
}
```

Eşitlik İşleci

C# dilinde iki farklı eşitlik türü vardır: referans eşitliği ve değer eşitliği. Değer eşitliği, eşitliğin yaygın olarak anlaşılan anlamıdır: iki nesnenin aynı değerleri içermesi anlamına gelir.

```
object a = new object();
object b = a;
System.Object.ReferenceEquals(a, b); //true

Console.WriteLine((2 + 2) == 4); // değer eşitliği: true
object s = 1;
object t = 1;
Console.WriteLine(s == t); //referans eşitliği, farklı nesneler: false

string a = "hello"; //const
string b = String.Copy(a); //instance
string c = "hello"; //const
Console.WriteLine(a == b); // metin değerlerini karşılaştırma: true

Console.WriteLine((object)a == (object)b); //metin referanslarını karşılaştırma: false
Console.WriteLine((object)a == (object)c); //metinlerin ikisi de sabit: true
```